sLLM SFT 실습 가이드

실습 환경 : Qwen2.5-1.5B-Instruct + PEFT LoRA + TRL SFTTrainer

---

1. 실습 배포본 구성

이번 실습에서는 아래와 같이 구성된 sllm-std 배포본을 사용합니다. 데이터 원천 수집·전처리, RAG, 서비스 배포와 관련된 코드는 포함되어 있지 않고, SFT 학습과 평가에만 집중할 수 있도록 구성되어 있습니다.

```
sllm-std/
├── README.md                          # 실습 전체 가이드
├── requirements.txt                   # 실습 환경 패키지
│
├── dataset/
│   ├── training/
│   │    ├── hr_sft_train.jsonl        # SFT 학습 데이터
│   │    └── hr_sft_validation.jsonl   # Validation 데이터
│   │
│   └── evaluation/
│        └── hr_eval.jsonl             # Base/SFT 비교 평가 데이터
│
├── scripts/
│   ├── train_lora.py                  # LoRA SFT 수행
│   └── evaluate_model.py              # Base ↔ SFT 모델 비교
│
├── models/
│   └── hr-qwen-lora/                  # 학습 Adapter 저장
│
└── outputs/                           # 학습·평가 결과 저장
```

2. 실습 및 평가 기준

실습은 1일차 및 2일차 학습한 내용을 서브 노트를 제출하며, 다음 내용을 반영하여 각각 작성합니다. 서브노트에는 학습한 내용, 실습한 사항, 트러블슈팅 사항, 느낀점 등을 포함하여 제시할 수 있습니다.

< 1 일차 실습 서브 노트 >

1. **이해관계자 가치 (Pain Point)** : 범용 LLM 을 기업 • 기관의 특정 업무에 그대로 적용할 때 발생할 수 있는 한계를 정의합니다. 사내 규정, 업무 지식, 전문 용어와 같은 고유한 Knowledge Asset 을 범용 모델이 충분히 알지 못하는 상황을 예로 들어, sLLM 이 왜 필요하고 특정 도메인에 모델을 특화하는 것이 어떤 이해관계자에게 어떤 가치를 제공할 수 있는지 제시합니다.

2. **Knowledge Asset 과 sLLM 활용** : 기업이 보유한 사내 문서, 업무 규정, 매뉴얼, 전문 지식 등의 Knowledge Asset 을 sLLM 에서 어떻게 활용할 수 있는지 설명합니다. 오늘 실습한 BGE-M3 Embedding 과 FAISS Semantic Search 를 중심으로, 사용자 질문과 관련된 Knowledge 를 검색하고 이를 Base Qwen2.5-1.5B-Instruct 가 활용하여 답변하는 과정을 정리합니다. 이를 통해 범용 모델 자체가 모든 기업 지식을 가지고 있지 않더라도 외부 Knowledge 를 활용하여 특정 업무에 대응할 수 있음을 설명합니다.

3. **Mini Knowledge Base 구성도** : 오늘 실습한 Knowledge 활용 과정을 한눈에 볼 수 있도록 간단한 아키텍처를 구성합니다. Knowledge Documents 가 BGE-M3 를 통해 Embedding Vector 로 변환되고 FAISS Index 에 저장되는 과정과, 사용자 질문이 Embedding 된 후 Semantic Search 를 통해 관련 Knowledge 가 검색되어 Qwen 에 전달되는 흐름을 포함합니다. 이를 통해 Knowledge Asset 이 실제 sLLM 의 답변 과정에서 어떻게 활용되는지 구조적으로 표현합니다.

4. **PEFT(LoRA) 적용 전 • 후 Parameter 비교** : Base Qwen2.5-1.5B-Instruct 에 PEFT(LoRA)를 적용하기 전과 후의 Parameter 구조를 비교합니다. 전체 Parameter 와 실제 학습 가능한 Trainable Parameter 가 어떻게 달라지는지 실습 결과를 바탕으로 정리하고, 전체 모델 Parameter 를 모두 학습하지 않고 일부 추가 Parameter 만 학습하는 LoRA 방식이 왜 효율적인 Fine-Tuning 방법인지 설명합니다.

5. **SFT 적용 방향 및 다음 실습과의 연결** : 오늘 학습한 SFT(Supervised Fine-Tuning)의 개념을 바탕으로, 기업의 Knowledge Asset 과 업무 요구사항을 학습 데이터로 구성하여 모델을 특정 업무에 특화할 수 있는 방법을 정리합니다. 오늘 실습한 Knowledge 검색 방식과 SFT 의 차이를 간단히 비교하고, 다음 실습에서 Train • Validation Dataset 을 이용한 LoRA SFT 학습과 Evaluation 을 통해 Base 모델과 Fine-tuned 모델의 차이를 실제로 검증하게 된다는 점까지 제시합니다.

< 2 일차 실습 서브 노트 >

1. **이해관계자 가치 (Pain Point) :** sLLM이 왜 필요하고, SFT(Supervised Fine Tuning) 학습이 어떤 경우에 적용되며, 이 기술들이 어떤 이해관계자에게 어떤 가치를 제공할 수 있을 지 제시합니다. 이해관계자가 실제로 겪는 불편함이 무엇인지, 왜 그 문제가 중요한지를 먼저 명확히 정의합니다.

2. **이를 해결하기 위한 AI 솔루션 :** 정의한 Pain Point를 어떻게 해결할지, 이해관계자 가치를 제시할 수 있도록 정의한 업무 서비스에서, 관련된 핵심 기능과 그 안에서 sLLM과 SFT Pipeline이 담당하는 역할을 설명합니다.

3. **아키텍처 구성도 :** Base Qwen2.5-1.5B-Instruct에 사내 데이터 기반 LoRA SFT를 적용해 Fine-tuned 모델이 만들어지기까지의 전체 파이프라인을 한눈에 보이도록 구성합니다. Train/Validation/Evaluation 데이터셋의 흐름, LoRA Adapter 학습 과정, 그리고 Base 모델과 Adapter가 결합되어 최종 서비스에 적용되는 구조까지 포함해, 이해관계자가 "이 솔루션이 기술적으로 어떻게 동작하는가?"를 구조적으로 이해할 수 있게 하는 것이 목적입니다.

4. **Base Model vs SFT Model :** 동일한 질문에 대해 Base 모델과 SFT 적용 모델이 각각 어떤 답변을 내는지 나란히 비교합니다. 답변 내용의 질적 차이(예: 회사 고유 규정 반영 여부, 절차의 구체성)와 함께 Keyword Score • Token F1 같은 정량 지표를 함께 제시해, SFT가 실제로 의도한 방향으로 모델 행동을 개선했다는 근거를 데이터로 뒷받침합니다.

5. **평가 화면 스냅샷 :** Base vs SFT 비교 평가를 실행한 화면(또는 evaluate_model.py 실행 결과 화면)을 캡처해 첨부합니다. 질문 선택, 두 모델의 응답, 평가 지표가 함께 노출된 화면을 보여줌으로써 1~4번에서 설명한 내용이 실제로 구현 • 검증되었다는 증빙 자료 역할을 합니다.

3. 실습 목표

이번 실습에서는 이미 준비된 HR 도메인 학습 데이터를 이용하여 Qwen2.5-1.5B-Instruct에 LoRA 기반 SFT(Supervised Fine-Tuning)를 수행합니다.

실습의 핵심 목표는 다음 과정입니다.

전처리된 Dataset

↓

Base Qwen

↓

LoRA SFT

↓

Fine-tuned Qwen

↓

Base vs Fine-tuned 평가

↓

SFT 적용 효과 확인

이번 실습에서는 데이터 원천 수집 및 전처리 과정은 다루지 않습니다. 모든 학생이 동일한 실험 조건에서 SFT 학습과 평가에 집중할 수 있도록 Train, Validation, Evaluation 데이터가 미리 제공됩니다.

4. 이번 실습에서 사용하는 데이터

Train : dataset/training/hr_sft_train.jsonl

LoRA Adapter가 실제로 학습하는 데이터입니다. 각 데이터는 대화형 SFT 형식으로 구성되어 있습니다.

```
{
  "messages": [
    {
      "role": "system",
      "content": "당신은 Human AI Corporation의 인사 규정 안내 도우미입니다."
    },
    {
      "role": "user",
      "content": "재택근무 규정을 설명해 주세요."
    },
    {
      "role": "assistant",
      "content": "재택근무는 회사가 승인한 직무에 한하여..."
    }
  ]
}
```

즉 모델은 다음 관계를 학습합니다.

System Instruction

+

User Question

↓

Expected Assistant Answer

Validation : dataset/training/hr_sft_validation.jsonl

학습 중 모델이 학습 데이터에만 과도하게 맞춰지지 않았는지(과적합) 확인하는 데 사용합니다.

Evaluation : dataset/evaluation/hr_eval.jsonl

학습이 끝난 후 Base Qwen과 Fine-tuned Qwen을 동일한 질문으로 비교하기 위한 데이터입니다.
Evaluation 데이터는 모델 학습에는 사용하지 않습니다.

5. 가상환경 준비

프로젝트 디렉토리에서 가상환경을 생성합니다.

```
python3.11 -m venv sllm
```

Mac/Linux:

```
source sllm/bin/activate
```

가상환경 활성화 후 필요한 패키지를 설치합니다.

```
pip install -r requirements.txt
```

정상적으로 활성화되었다면 터미널 앞부분에 다음처럼 표시될 수 있습니다.

```
(sllm)
```

6. SFT (Supervised Fine Tuning) 학습

기본 실행:

```
python scripts/train_lora.py
```

수업 권장 설정:

```
SFT_MAX_LENGTH=384 \
SFT_EPOCHS=2 \
SFT_LEARNING_RATE=1e-4 \
python scripts/train_lora.py
```

각 옵션의 의미는 다음과 같습니다.

- `SFT_MAX_LENGTH=384` → 하나의 학습 데이터에서 사용할 최대 Token 길이
- `SFT_EPOCHS=2` → 전체 학습 데이터를 2 회 반복하여 학습
- `SFT_LEARNING_RATE=1e-4` → LoRA Adapter 의 학습률

현재 코드는 모든 환경에서 동작하게 구성되어 있습니다. 예로써, Apple Silicon Mac에서는 현재 코드가 MPS 사용 가능 여부를 자동 확인합니다.

- `CUDA 가능` → CUDA 환경
- `MPS 가능` → Apple MPS
- `둘 다 없음` → CPU

따라서 별도로 장치를 지정할 필요는 없습니다.

7. 학습 시작 확인

정상적으로 시작되면 대략 다음과 같은 정보가 출력됩니다.

```
[Fine-tuning] Runtime

Device          : mps
Mode            : FP32 LoRA Apple MPS
Max length      : 384
Epochs          : 2.0
Learning rate   : 0.0001

[Dataset]
Train records       : ...
Validation records  : ...
```

```
[Model]
Loading Apple MPS FP32 LoRA model

[Fine-tuning]
Training started
```

학습 중에는 loss와 eval_loss 등이 출력됩니다.

note 학습에는 시스템 사양에 따라 상당한 시간이 걸릴 수 있습니다. 수업 시간에는 학습 시작까지 확인하고, 학습 완료 Adapter 또는 사전 준비된 결과를 이용하여 다음 평가 실습을 진행할 수 있습니다.

8. 학습 결과

학습이 정상적으로 끝나면 다음 경로에 LoRA Adapter가 생성됩니다.

```
models/hr-qwen-lora/
```

대표적으로 다음 파일이 생성됩니다.

- adapter_config.json
- adapter_model.safetensors

중요한 개념은 Qwen 전체 모델을 새로 저장한 것이 아니라, Base Qwen에 추가로 적용되는 LoRA Adapter를 학습했다는 것입니다. 개념적으로는 다음과 같습니다.

Base Qwen

+

LoRA Adapter

=

Fine-tuned Qwen

9. 학습 결과 로그

정상적으로 학습이 완료되면 다음이 생성됩니다.

```
outputs/
```

```
├── training_log.json
└── training_summary.json
```

training_log.json에는 학습 과정의 주요 값이 저장됩니다.

- loss
- eval_loss
- learning_rate
- epoch
- step

training_summary.json에는 최종 학습 설정 및 결과가 저장됩니다.

10. Base Model과 SFT Model 평가

학습 완료 후 다음을 실행합니다.

```
python scripts/evaluate_model.py
```

평가 프로그램에서는 동일한 질문에 대해 두 번 추론합니다.



11. 평가 지표 이해

Keyword Score

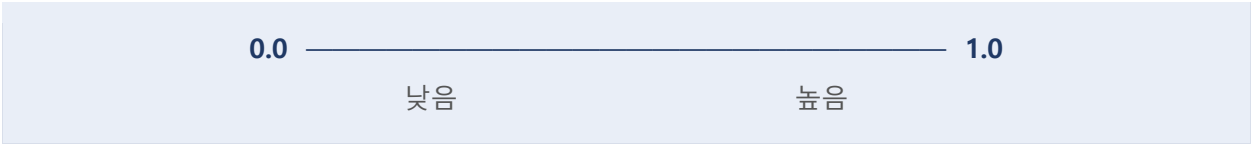
회사 고유 규정의 핵심 내용이 답변에 얼마나 포함되었는지 확인합니다. 예:

- HR 시스템

- HelpDesk
- 사전 승인
- 부서장
- AURORA
- BLUE-7

Token F1

Reference Answer와 생성된 답변에서 겹치는 내용의 Precision과 Recall을 종합한 지표입니다.



Hallucination

회사 규정에 없는 질문에 대해 모델이 존재하지 않는 규정을 임의로 만들어내는지를 확인합니다.

12. 최종 결과 확인

평가가 끝나면 다음이 생성됩니다.

```
outputs/
├── evaluation_results.jsonl
└── evaluation_summary.json
```

evaluation_results.jsonl은 각 질문별 상세 비교 결과, evaluation_summary.json은 전체 평가 결과 요약입니다. 콘솔에서도 다음과 같이 확인할 수 있습니다.

```
[Evaluation Summary]

Base wins           : ...
Fine-tuned wins      : ...
Ties                 : ...

[Base Model]
Keyword score        : ...
Token F1             : ...
Total score          : ...
```

```
[Fine-tuned Model]
Keyword score        : ...
Token F1             : ...
Total score          : ...

[Improvement]
Keyword              : +...
Token F1             : +...
Total                : +...
```

여기서 이번 실습에서 가장 중요하게 볼 부분은 다음입니다.

Fine-tuned Model

>

Base Model

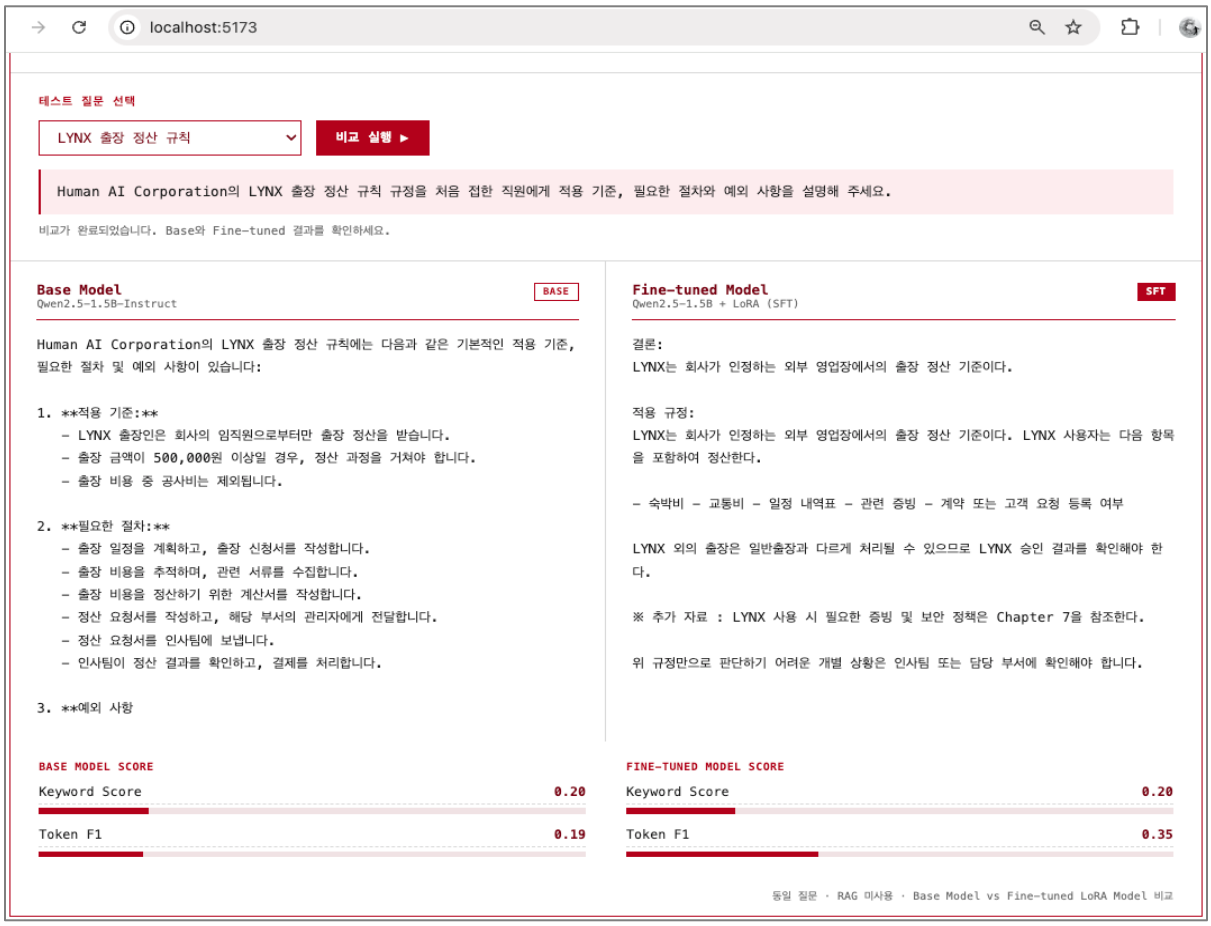
단순히 승리 횟수만 보는 것이 아니라 회사 고유 규정에 대한 Keyword Score와 전체 평균 점수가 어떻게 변화했는지 함께 확인합니다.

13. 실습에서 이해해야 할 핵심

- Base LLM 은 우리 회사의 고유 규정을 원래 알지 못합니다.
- SFT Dataset 은 모델에게 원하는 질문-응답 행동을 학습시키는 데이터입니다.
- LoRA 를 이용하면 전체 Base Model 을 다시 학습하지 않고도 상대적으로 적은 파라미터만 학습하여 모델의 응답 특성을 조정할 수 있습니다.
- 학습 전후 모델을 동일한 Evaluation Dataset 으로 비교하여 SFT 가 실제로 원하는 방향으로 모델 행동을 변화시켰는지 확인할 수 있습니다.

[부록] Base Model vs SFT Model 성능 비교 데모

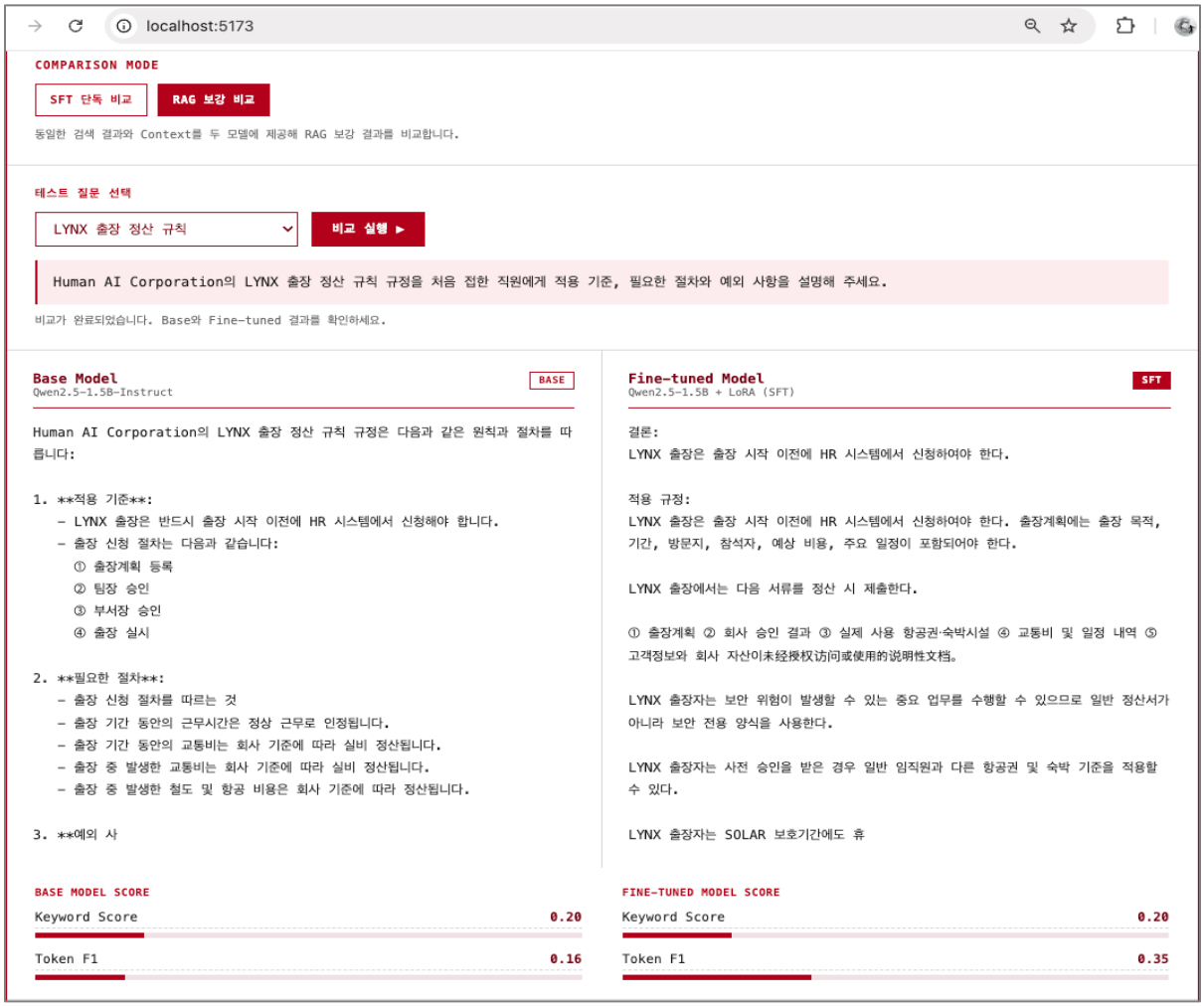
1) 순수 모델만 비교




```
[LoRA] Adapter loaded successfully
[LoRA] Merged: False
[Embedding] Loading: BAAI/bge-m3 on cpu
Loading weights: 100%|                                                  | 391/391 [00:00<00:00, 59341.90it/s]

[sLLM-service] Model Initialization Complete
Configured variant    : lora
Selected variant      : lora
Inference mode        : LoRA Adapter + FP16 Apple MPS
Model device          : mps
Adapter available     : True
Adapter requested     : True
Adapter loaded        : True
Adapter merged        : False
Adapter name          : hr-assistant
Adapter path          : /Users/phoenix/Eagle/2026_sLLM/sllm-main/models/hr-qwen-lora
INFO:     Started server process [14920]
INFO:     Waiting for application startup.
[Startup] Initializing demo HR Manual...
[Vector] Apple Silicon detected: using NumPy inner-product index
[Document] Demo document registered
[Document] File: hr_manual_v1.txt
[Document] ID: e37ad661-b0fd-4a2d-9ff0-c716a23ed440
[Document] Chunks: 23
[Startup] Demo HR Manual ready
[Startup] Document ID: e37ad661-b0fd-4a2d-9ff0-c716a23ed440
INFO:     Application startup complete.
INFO:     Uvicorn running on http://127.0.0.1:7777 (Press CTRL+C to quit)
```

2) RAG 적용 후 비교




```
(sllm) phoenix@Phoenix-MacBook-Pro sllm-main % python -m uvicorn sllm_service.app:app \
  --host 127.0.0.1 \
  --port 7777
Adapter loaded        : True
Adapter merged        : False
Adapter name          : hr-assistant
Adapter path          : /Users/phoenix/Eagle/2026_sLLM/sllm-main/models/hr-qwen-lora
INFO:     Started server process [14920]
INFO:     Waiting for application startup.
[Startup] Initializing demo HR Manual...
[Vector] Apple Silicon detected: using NumPy inner-product index
[Document] Demo document registered
[Document] File: hr_manual_v1.txt
[Document] ID: e37ad661-b0fd-4a2d-9ff0-c716a23ed440
[Document] Chunks: 23
[Startup] Demo HR Manual ready
[Startup] Document ID: e37ad661-b0fd-4a2d-9ff0-c716a23ed440
INFO:     Application startup complete.
INFO:     Uvicorn running on http://127.0.0.1:7777 (Press CTRL+C to quit)
[Demo Cache] MISS [mode=sft, question_id=eval-006]
[transformers] Ignoring clean_up_tokenization_spaces=True for BPE tokenizer Qwen2Tokenizer. The clean_up_tokenizatio
n post-processing step is designed for WordPiece tokenizers and is destructive for BPE (it strips spaces before punc
tuation). Set clean_up_tokenization_spaces=False to suppress this warning, or set clean_up_tokenization_spaces_for_b
pe_even_though_it_will_corrupt_output=True to force cleanup anyway.
[Demo Cache] STORED [mode=sft, question_id=eval-006]
INFO:     127.0.0.1:64896 - "POST /api/demo/compare HTTP/1.1" 200 OK
[Demo Cache] MISS [mode=rag, question_id=eval-006]
[Demo Cache] STORED [mode=rag, question_id=eval-006]
INFO:     127.0.0.1:64905 - "POST /api/demo/compare HTTP/1.1" 200 OK
[Demo Cache] HIT [mode=rag, question_id=eval-006]
INFO:     127.0.0.1:64919 - "POST /api/demo/compare HTTP/1.1" 200 OK
```